Chapter 10: File I/O

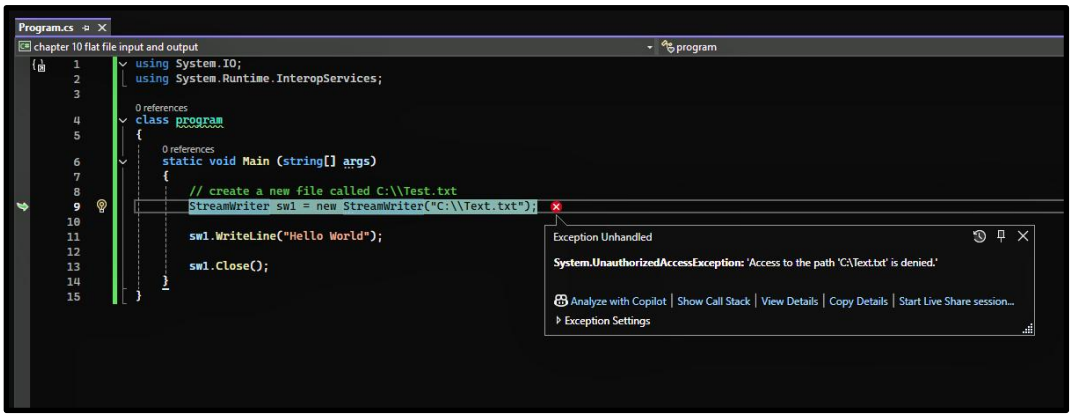
In this chapter we are going to take a look at flat file I/O, that is to say flat file input and output. Before the advent of XML, flat files were the standard way to temporarily store information. Now XML files (which we will study in chapter 22) are more commonly used for that purpose. Today, flat files are typically only used to store things like emails, spreadsheets, word processing documents, music and video. As such, we'll just learn how to open, write to, read from, close and check for the existence of a flat file.

Okay, time to jump to the inside and check out some code.

We begin by checking out the code shown here. (Figure 10.1) Please note the addition of the namespaces at the top of my code. I added the System.IO namespace and intellisense decided I needed the System.Runtime.InteropServices namespace.

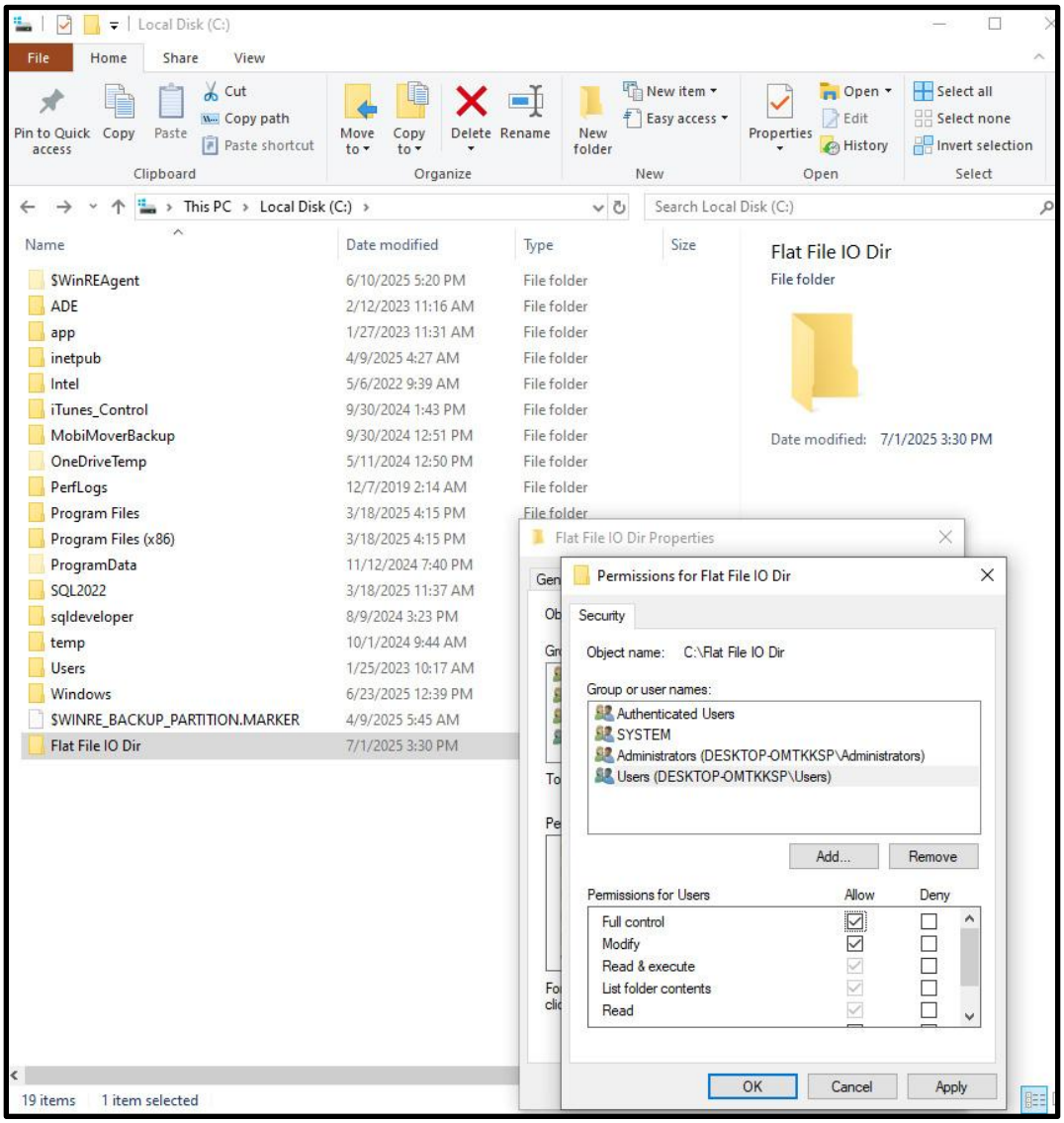
As shown in the figure, I attempted to create and open a file named Test.txt in the root directory of my C drive and I got an UnauthorizedAccessException. I knew this one would blow up, but I wanted to make a point. Visual Studio needs full control over any directory into which it writes a file.

Figure 10.1: UnauthorizedAccessException



So how do we correct this issue? Well in Windows, I simply created a new directory named C:\Flat File IO Dir and right clicked on that guy. One of the menu options is properties so pick that guy. One of the tabs will read Security. Click on that guy and you see the screen shown here. (Figure 10.2) Now all you need to do is give the Users group Full control of the new directory and you are ready to rock and roll.

Figure 10.2: New Directory With User Privileges



I now update my code to point at my newly created directory as shown here. (Figure 10.3) The results shown here seem to indicate that everything worked as expected. (Figure 10.4)

Several things warrant our attention in this code.

- First, note the use of the @ symbol prefix to my directory path. This is the verbatim symbol, and it means to take the following string text exactly as shown with no extra interpretation of the dot notation or slashes.
- Next, please note that I did not have to check for the prior existence of our Test.txt file because the StreamWriter class creates a new file if none exists or truncates the file if it previously existed.
- Finally, please note how I explicitly closed the file after writing to it.

Figure 10.3: StreamWriter Class

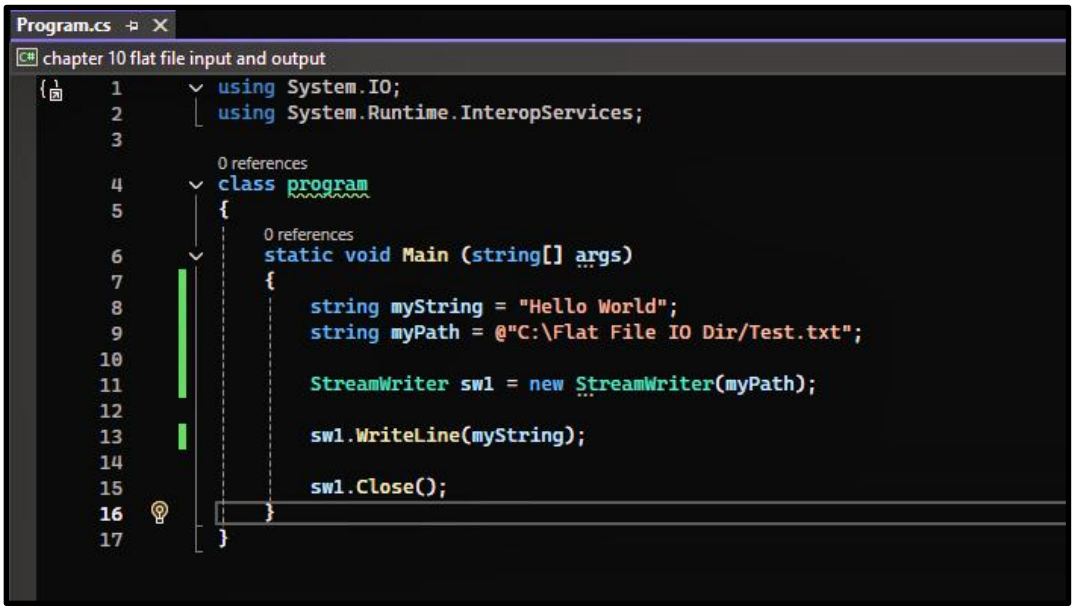
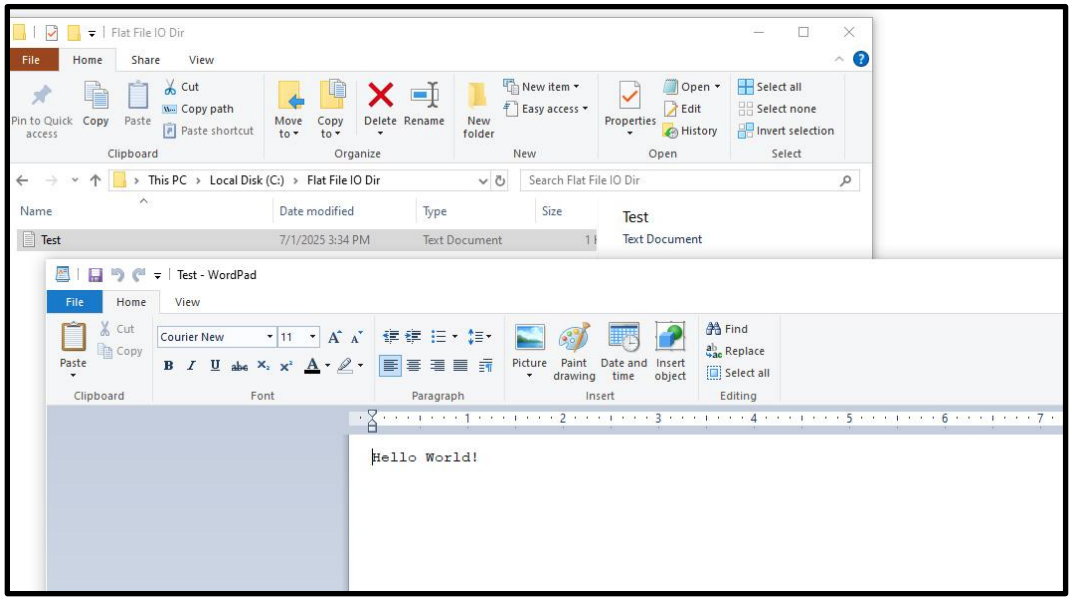


Figure 10.4: Output



In our next example we take a look at the FileStream and BinaryWriter classes. (Figure 10.5) Although we wrote out text in this example the same methodology works for the bytes making up an image, video or music. Please note that the using keyword ensures that the file is properly closed while the FileMode.OpenOrCreate enum ensures proper creation or opening of the file if it already exists and truncation of any previous contents. Come on, that is pretty elegant code for only a few lines. The output from our new code is shown here. (Figure 10.6)

Figure 10.5: FileStream and BinaryWriter Classes

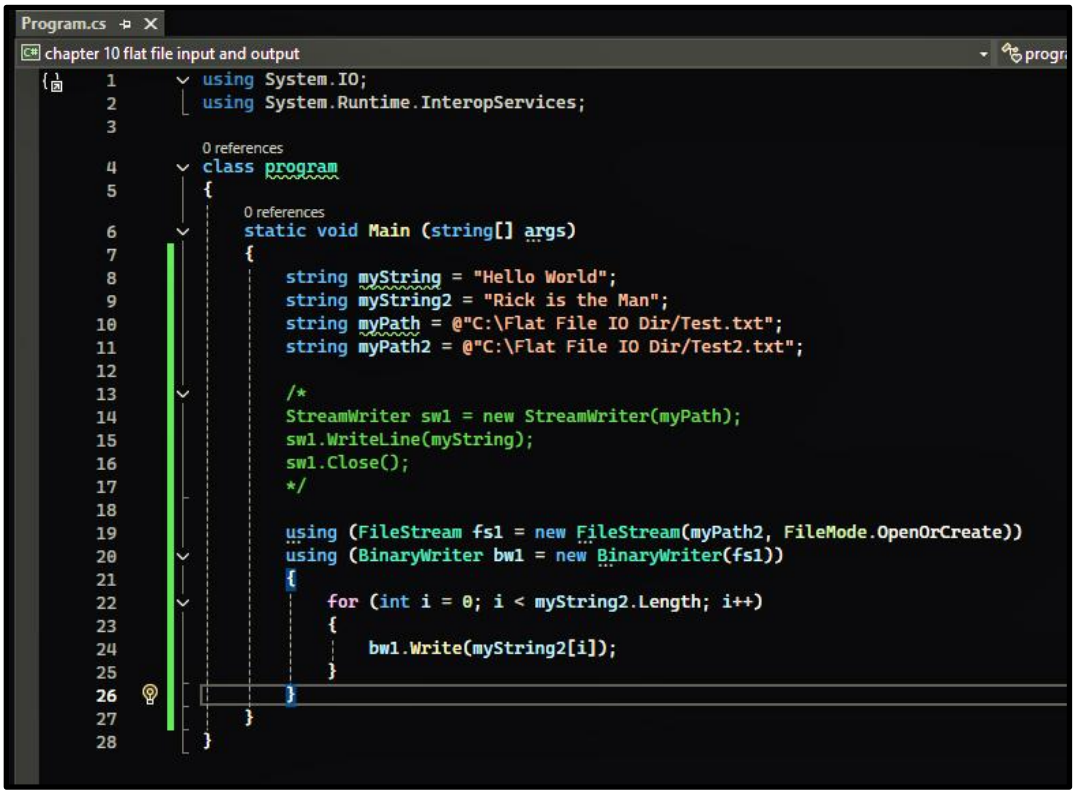
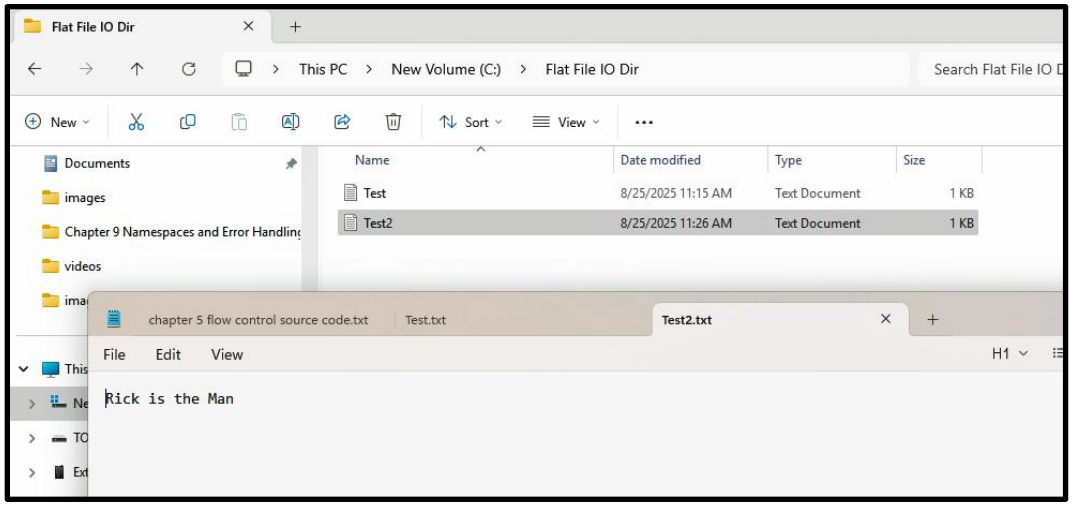
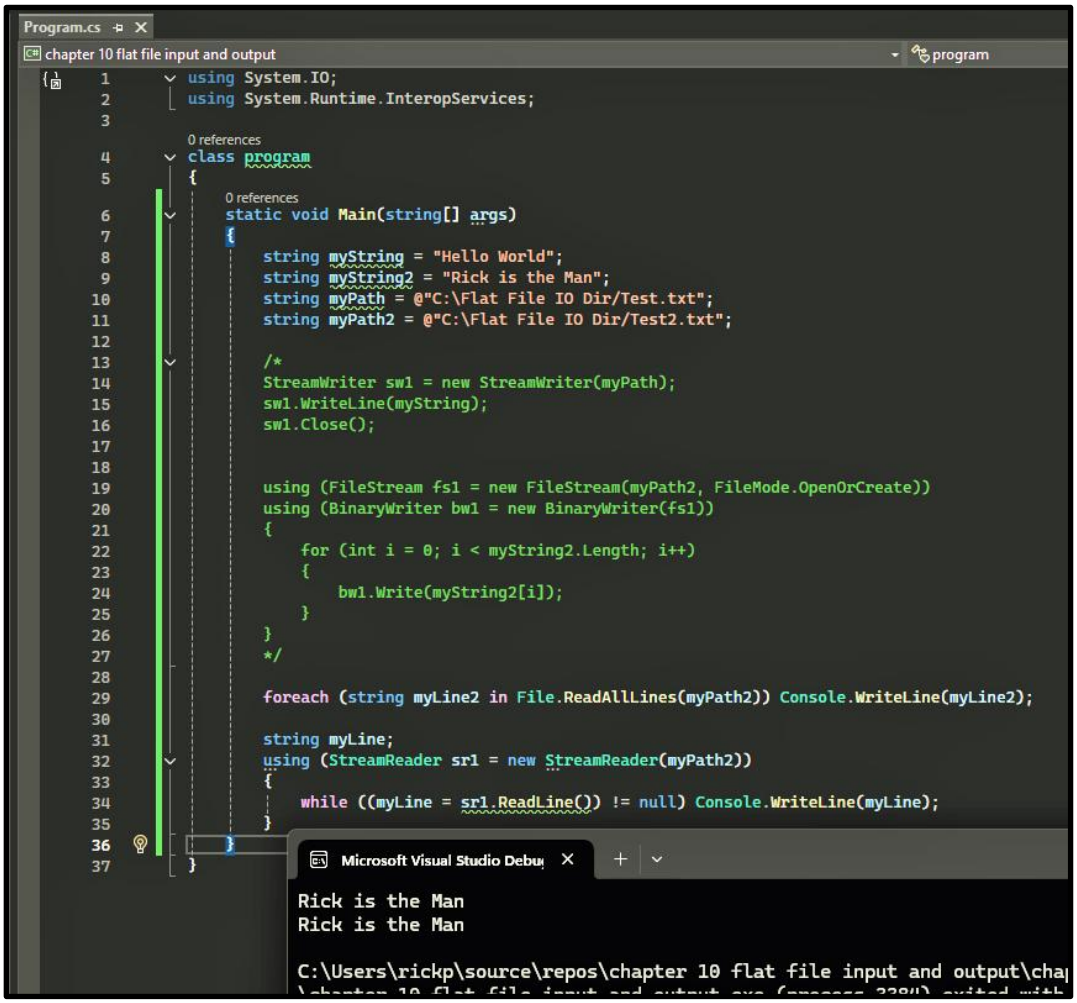


Figure 10.6: Output



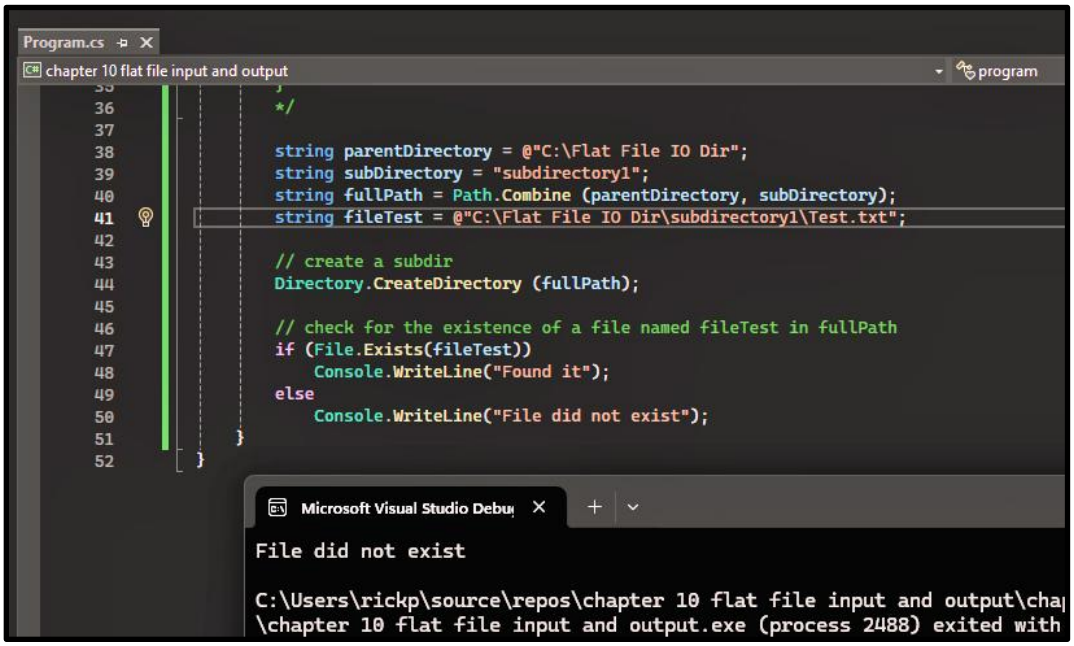
Time now to read this information back into our code. This image (Figure 10.7) shows the use of the File class ReadLines method and the StreamReader class. The StreamReader class is more modern so it is probably your preferred methodology, but you will still encounter a ton of code out there which utilizes the ReadLines method so it's good to know about both.

Figure 10.7: File.ReadLines and StreamReader



One more quick example and we can put a bow on flat file I/O. In this example (Figure 10.8) we demonstrate how to create a subdirectory and check for the existence of a specific filename. Pretty straight forward stuff.

Figure 10.8: Subdirectory Creation and File Existence



This concludes our discussion of flat file I/O and also our discussion of the basics of the C# programming language. In our next section we start looking at the graphic components of C#. Should be great fun.